

Wireless Environmental Sensing System

[W.E.S.S]

**Joshua Collier, Jesse Mistr, Logan Maxwell, Andrew Garzon, Zane
Hernke, Nahum Cortezzo**

**Dept. of Electrical and Computer Engineering, Box 352500
Paul Allen Center, 185 E Stevens Way NE AE100R, Seattle, WA 98195
University of Washington
Seattle, Washington 98195-2500 U.S.A.**

©

Rev. 1 March 12, 2025

Rev. 2 March 14, 2025

Rev. 3 March 16, 2025



**ELECTRICAL & COMPUTER
ENGINEERING**

UNIVERSITY *of* WASHINGTON

Introduction

The Wireless Environmental Sensing System (W.E.S.S) project was started to create a scalable, cost-effective solution for monitoring environmental quality in both urban and rural settings. With a growing global emphasis on environmental sustainability, our project aims to empower communities, researchers, and public health agencies by delivering real-time data on air and water quality. The W.E.S.S network utilizes a nodal mesh architecture in which a parent node communicates with numerous child nodes that capture air pollutant levels (such as NH_3 , NO_2 , CO) and water quality parameters (TDS and turbidity), both of which are easily interchangeable with minimal effort. This project not only addresses immediate monitoring needs but also lays the groundwork for long-term data trend analysis that can inform policy decisions and conservation efforts.

Our system was designed to be simple enough for hobbyists to deploy while remaining robust for municipal-scale operations. By leveraging industry-standard hardware platforms (STM32F4 Discovery and Raspberry Pi 4) and compliant wireless technologies (LoRa modules), W.E.S.S strikes a balance between advanced functionality and ease of use. This report outlines our complete development process—from requirements and constraints to experimental validation, outcomes, and recommendations for future work.

Team, Roles, and Responsibilities

Hardware & Data Acquisition Subteam:

- Joshua
 - Acting project manager (tracked deadlines, budget constraints, assigned tasks, and directed team efforts to tackle critical challenges.
 - Procured project materials, managed budget, and placed orders on project peripherals and sensors
 - Led peripheral sensor integration with the STM32 and built up major software components to facilitate data acquisition in line with project goals.
 - Led peripheral sensor integration with a secondary simulated child node for scalability "proof of concept".
 - Coordinated major project goals and deliverables and spurred team communication and collaboration across structural groups.
- Zane
 - Full hardware implementation of all sensors into a consolidated package, including debugging hardware for any errors.

- Design of 3d printed Project box with a focus on mass production for a large-scale operation.
- Worked closely with the firmware/software team, ensuring proper connections between hardware and software components.

Wireless Transmissions Subteam:

- Nahum
 - Compared different wireless options before deciding on LoRa modules for the project
 - Helped debug the LoRa module transmission code in IAR workbench
 - Reformatting of data to be transmitted
- Andrew
 - Configured UART ports for transmission of data between LoRa modules and the Pi/STM32.
 - Wrote code for how the modules would transmit data and be configured in the local mesh network.
 - Provided USB serial dongle for easy and seamless debugging.
 - Authored Python scripts for gathering and logging data from the two child nodes.

Data Analysis & Visual Displays Subteam:

- Logan
 - Procured and attached Raspberry Pi to the touchscreen interface.
 - Wrote the initial Python code that provides the HTML content for each tab of the front end.
 - Wrote the startup script that launches the frontend web server, data collection script, and web browser with one click.
- Jesse
 - Design lead for front end, *i.e.* page layout and theming
 - Researched Python packages for front-end display
 - Refined initial tab content code, organized Python scripts for app code
 - General web page layout debugging and style updates

Product Requirements Document (PRD)

W.E.S.S is a wireless nodal mesh network comprised of both child node devices and parent node devices that are connected in a point-and-spoke format where the child nodes represent the spokes, and the parent nodes represent the points. **Figure [1]** visualizes this structure.

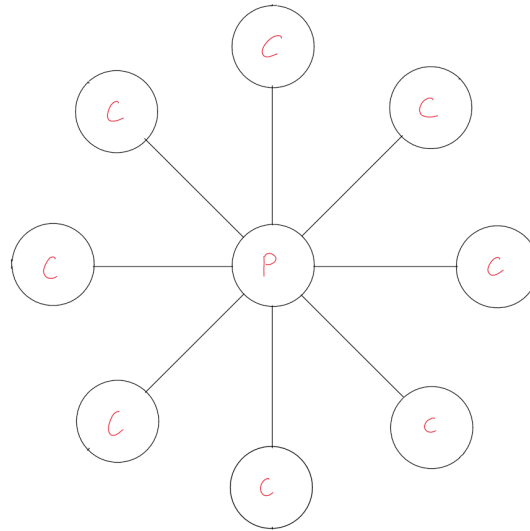


Figure [1]:

Illustration of the point-and-spoke structure of the W.E.S.S system. Represents a sample parent node and the C's represent sample children nodes

The W.E.S.S network periodically samples environmental quality data including common air pollutants NH₃, NO₂, and CO, as well as common water quality parameters like total dissolved solids (TDS), and turbidity (TURB) levels.

The need for the W.E.S.S system arises from the global trend of communities and developing cities being more interested in becoming environmentally conscious and decreasing the extent to which their everyday activities negatively impact their environment. This need is seen particularly among conservationist organizations and agencies seeking to preserve national or communal wild spaces such as parks, forests, lakes, and other ecospheres. The W.E.S.S system also offers valuable information to public health agencies seeking to identify communities that are more at risk than others for health complications pertaining to prolonged exposure to air or water pollutants.

W.E.S.S is a network intended to be deployed in outdoor environments where environmental monitoring is desired around a central point of interest such as city centers, ranger outposts at national parks, recreation centers, industrial sectors, farmlands, local communities, and more.

This system is designed to be simple enough to be used by anyone with an interest in monitoring environmental quality data at any intensity level. Our system is adaptable to the

hobbyist collecting data surrounding their home with only a few nodes to cities seeking to gather comprehensive environmental data of their territories using up to 65,536 nodes.

The W.E.S.S network must be used with a minimum of at least one parent node, and one child node. It is utilized by securing the parent node in an easily accessible focal point where data accumulation, visualization, and processing are desired. This is typically within a home, research lab, park maintenance building, ranger stations, city centers, and other places that could harbor the parent node where someone or multiple people could interact with it. The child node(s) would then be deployed in the geographical region of interest where environmental monitoring is desired. The child node(s) should not exceed a maximum distance of 20km from the parent node as this is the absolute maximum transmission range under ideal conditions and minimal interference, for more reliable operation a distance of 0 - 15km should be used.

The setup for this system is simply plug and play, as each child node and parent node is pre-configured and ready to go as long as there is a power source for the child node(s) (size dependent on desired maintenance frequency) and a power source and an internet connection for the parent node. All critical functions still work (healthy range node coloring, time series graphs, and download functionality over local hosting) even if there is no available internet connection for the parent node. A map overlay for the region of interest is available to display relative geographical information per child node if an internet connection is available. Over time the W.E.S.S network will compile data related to the region in which the child nodes are deployed and provide information about historical data trends corresponding to environmental pollutants. These data trends are intended to be used in tandem with the user's environmental goals whether that be informing governmental policy around city development and industrial activities, providing quality data to identify communities or regions that are exposed to more pollutants than others, and supplementing public health tracking, or tracking environmental risk factors in valuable ecospheres.

The W.E.S.S network is designed to operate in tandem with a variety of supporting components that ensure its functionality. The system relies on stable power supplies, with child nodes operating on dedicated low-voltage sources (typically 3–5V) and the parent node requiring a steady 5–6V supply, often backed by mains or battery power to guarantee uninterrupted performance.

In parallel, the network is integrated with various hardware and software elements to provide a comprehensive monitoring solution. It works with a range of environmental sensors for detecting air pollutants and water quality parameters, and it utilizes pre-configured LoRa wireless modules to facilitate long-range communication between nodes. The hardware components are housed in weather-resistant enclosures to withstand outdoor conditions, while the user interface may include a touchscreen or monitor for local interaction. Furthermore, the system supports integration with web-based visualization tools and mapping systems, enabling users to analyze real-time and historical data trends. This cohesive setup not only simplifies deployment but also extends the utility of W.E.S.S across diverse applications, from personal environmental monitoring to large-scale public health initiatives.

Realistic Constraints and Engineering Standards

Project Constraints:

Some of the design constraints relevant to this engineering project include the necessity to utilize the STM32 F4 Discovery microcontroller and the Raspberry Pi 4, a set 10-week deadline to provide a minimum viable product, and the need to keep all R&D and prototyping costs below \$300. These project constraints incentivized us to keep our designs simple and straightforward so that we could remain under budget constraints and still provide a reasonable and functional product at the end of 10 weeks.

Child Node Constraints:

- Child nodes must be compact and contained within a weather-resistant housing.
- Child nodes must be supplied with an external power source in the range of 3-5V.
- Child nodes only have access to 192 Kb of RAM.
- Child node on-chip flash memory for storing autonomous programs is 1Mb.
- Child nodes must be programmed in C or C++ using IAR workbench or STM32Cube IDE.
- Child node programs must not exceed 1 Mb in size.
- The child node may sample environmental data many times (not exceeding 192Kb) to calculate an effective average, but must only send out an averaged set of data to the parent node once every hour to meet time series goals without exceeding parent node memory capacities before a useful time series is generated (several weeks/months).
- The accuracy of analog sensor readings is limited by on-chip 12-bit resolution (4096 bit) ADC's which offer granularity of about 2.44mv per step and may impact measurement accuracy.
- Depending on the operating environment, the child node must be within ~0-15 km from the parent node to effectively and reliably transmit data.

The constraints set forth by the child node influenced our decisions to program the child nodes in such a way that they could collect a series of data points over time and average them roughly every hour before sending the data to the parent node. This would help smooth out some of the inaccuracies presented by the limited ADC resolution by averaging multiple data points over time and would also allow our system to make efficient use of the long-term memory present within the parent node when collecting time series data for historical trendline generation. Additionally, due to the limited flash memory available on the child node microcontroller we took some time to refactor our program to make it smaller, and more memory efficient wherever possible. The sensitive nature of the electrical devices integrated with the child node also prompted the need to develop a secured housing for the child node microcontroller and peripherals. This housing keeps external environmental debris away from sensors and other sensitive equipment while also providing a measure of user protection against static discharge and electrical shock. Another feature of the W.E.S.S network influenced by the constraints set forth by the child node is the configuration of the REYAX RYLR988 LoRa communication

module. In order to provide a robust, reliable, and effective data stream between child nodes and the parent node over an effective distance that would make sense for the W.E.S.S network, the RYLR998 LoRa modules were configured to prioritize long-range transmissions at lower bit rates since data would only be sent to the parent node on an hourly basis. An integrated feature of the RYLR998 that influenced our choice to use this module in our design is its high interference immunity that would assure reliable long-distance communication even in environments that have a moderate level of RF interference.

Parent Node Constraints:

- Parent node must be supplied with external power in the range of 5-6V.
- Parent node must be indoors or in a shaded environment in order to see the screen.
- Parent node programs and stored data may access 10GB of flash storage after the Raspbian operating system is installed.
- Parent node programs must not exceed 4GB of RAM usage.
- Parent node must be connected to the internet for displaying the map of child nodes.
- A USB mouse is needed to double click the startup script (afterwards only touchscreen is necessary).
- Parent node must always be listening for incoming data from child nodes due to limited timekeeping accuracy of child node hourly transmission.
- Webpage content must take a reasonable amount of time to load, so CSV file(s) must not get too large.

With the goals of our project, most of the parent node constraints would not serve as limiting factors to our project. Since the main functions of the Raspberry Pi were to serve a web page layout and store data from the child nodes, we were mostly concerned with the available long term memory and short term memory. The biggest factor would be ensuring our program does not exceed the 4GB of RAM constraint. The amount of RAM required to serve a webpage is not much, so we would not encounter this issue here. However, the program does read and parse a CSV (comma separated value) file. If this file is very large, it could potentially result in program crashes due to exceeding the RAM limitation. To combat this, we made sure to communicate with the Wireless and Data Acquisition teams about the period between data transmissions so that the CSV would slowly accumulate values. We decided that there would be a transmission every hour. This period of time resulted in a CSV that would gain at most 24 data points in a day per node, and even over the course of a month, the CSV file would not exceed 1000 values. Since CSV files are text files the encoded data does not take up a lot of memory, so a 1000 line CSV would only be a few kilobytes of data. However, if we transmitted data every minute, we would get over 1000 lines in one day, and this would not be sustainable if we left the device to transmit over the course of a month.

ENGINEERING STANDARDS:

Wireless communication standards

LoRaWAN - Regulations for LoRa-based communications modules

- The W.E.S.S network must be compliant with the rules and regulations pertaining to these types of wireless communication devices. The REYAX RYLR998 LoRa module complies with these standards and by a consequence, the W.E.S.S network also complies with them since all wireless communications are done through the use of the RYLR998. For all FCC regulations going forward, **note that our specific implementation of the RYLR998 makes use of a 915 MHz operating frequency to comply with FCC regulations.**

FCC RF human radiation exposure standard

- This standard regulates how much radiation exposure is acceptable for human users near operational W.E.S.S devices. The accepted standard is a 1.6 (SAR). The W.E.S.S network meets these standards by operating the radiator (REYAX RYLR998 LoRa module) within its specified operating conditions and frequency ranges. Additionally, the intended application of the W.E.S.S network keeps most primary sources of radiation outdoors generally away from direct contact with humans with the exception of the parent node.

FCC Part 15.205 Operation in restricted frequency bands

- The W.E.S.S network must abide by this regulation and either avoid transmitting data at the frequencies defined in this standard or do so below a specified power rating pertaining to the specific band of operation. The REYAX RYLR998 LoRa module used for wireless communications functions between 902-928Mhz and is beyond the scope of this regulation but it is still worth mentioning that improper use or system errors may cause interference with restricted bands and care should be taken to correctly and carefully configure the RYLR998 to operate in its intended frequency band.

FCC Part 15.247 - Unlicensed Spread Spectrum & Digital Transmission Devices

- This regulation sets standards for unlicensed transmitting devices operating in specific frequency bands (including LoRa bands) and how these devices should operate to maintain efficiency and no interference with licensed devices. W.E.S.S falls under this regulatory standard since the W.E.S.S network operates as a system of unlicensed transmitters using Chirp Spread Spectrum technology which falls under the category of Direct Sequence Spread Spectrum, one of the operational standards that FCC part 15.247 regulates and approves as long as transmission power does not exceed 1 watt. Proper setup and operation of the REYAX RYLR998 complies with this standard.

FCC Part 15.209 - Unwanted spurious emissions

- This regulation sets standards for unintentional frequency pollution and noise generation for frequencies outside of the intended band of operation for transmission devices. The W.E.S.S network must comply with this standard by making sure that frequencies generated outside of our intended bandwidth snuff out quickly with distance and stay below a limit of 500 $\mu\text{V/m}$ (50 dB $\mu\text{V/m}$) at 3 meters. This standard is complied with through proper use and configuration of the RYLR988.

Personal safety and durability standards

IEC 61140 - Protection Against Electrical Shock

IEC 60529 - Protection By Enclosure

IEC 60664 - Dust, Debris, and external pollutants

IEC 60721 - Environmental operating conditions

IEC 61010 - Safety requirements for measuring equipment

- This set of standards defines the requirements for a robust system that is safe to handle and that is resilient to the set of conditions under which it is intended to perform. The W.E.S.S system is designed to comply with these standards through the use of a custom-made weather-resistant case to protect the child node electronics from external debris and to serve as a barrier between users and electrical equipment. The parent nodes are also equipped with a touchscreen that facilitates all user interactions with the parent node and serves as the protective barrier between the user and the Raspberry Pi.

Environmental safety standards

RoHS - 173-337 WAC

- Although the W.E.S.S network can be deployed anywhere, users in the state of Washington must abide by the RoHS equivalent 173-337 WAC regulations that control what substances are considered hazardous in devices and products such as lead and mercury. W.E.S.S complies with this standard by using microcontrollers and peripheral devices that are RoHS-compliant.

Electromagnetic compatibility standards

IEC 61000 - Operational under expected electromagnetic conditions.

- This standard regulates the testing of electrical devices under the expected electromagnetic conditions for their intended use. This assures that devices are able to perform properly when used in their intended manner and are resilient to deviations from

these expected electromagnetic conditions. The W.E.S.S network, to assure proper functionality under expected electromagnetic conditions was tested both indoors and outdoors in a densely populated urban environment where RF interference is present. Additionally, our selection of the REYAX RYLR998 LoRa communication module supports our goals of building a resilient and reliable communication network through its high interference immunity that is built into the design of the chip.

Data storage standards

RFC 4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files

- Describes comma separated values (CSV) files by combining the most widely accepted standards for how they should be formatted. This memo is not technically a standard but is considered to be the most accepted description of CSV files.
- One deviation used for testing purposes was using a '#' character to delineate lines that are comments, and are therefore ignored when loading data.
- Another departure from the standard we implemented was allowing for some lines to have more fields than the header line does. Occasionally on data collection restart linking to an old CSV file, unneeded header information was written onto the last line of the file (which is presumed to be a sensor reading), then new sensor data was written to the next line. We simply ignore the extra fields whilst RFC 4180 advises against extra fields entirely.
- There exists no IEEE standard for CSV files.

WCAG 2.1 (WCAG 2.2 pending approval as ISO/IEC DIS 40500 as of March 2025)

- Web Content Accessibility Guidelines is a standard produced by the World Wide Web Consortium (W3C) to define guidelines for creating an accessible, international, private, and secure webpage. In the scope of the W.E.S.S. project, we mainly considered accessibility features since our web page does not track user data nor handle sensitive identifying information. Additionally, we did not develop our webpage with international standards in mind as this project was local, and during development we had no plans of going international. A couple of the accessibility guidelines we focused on are below.
- Guideline 1.41: Use of color
This guideline ensures that color is not the only distinguishing factor of information. We followed this guideline by ensuring that when color is used on either the map or trendline that there are text labels associated with the color spectrum, e.g. the color bar for data points on the map is a unique range for each pollutant, and the colors follow the

common cultural associations of green with good values, yellow with intermediate values, and red with bad values. Additionally, pollutants on the trendline are distinguished with colors. The colors are labeled on a legend so that the user can distinguish between pollutants.

- Guideline 1.4.6: Contrast (enhanced)

This guideline helps to ensure text is readable. There needs to be enough contrast between foreground (text) color and background color. The W.E.S.S. web page is designed around a dark theme, with a dark background and light text. The color pallet chosen for this project uses the hex code #272838 (Raisin Black) for the background, #D8D0C1 (Bone) for 'unimportant' body text, #f2f4ff (Ghost White) for 'important' body and header text and background contrast when using a dark font color, and #009B72 (Shamrock Green) for titles, major headers, and accent colors. W3C suggests a contrast ratio of at least 7:1 for body text, which we achieve with both Bone and Ghost White against Raisin Black. We did not meet the recommended contrast ratio with Shamrock Green, however. While large text has a lower ratio standard of 4.5:1, we only reached 4.1:1. Since this color was used more sparingly and more often as an accent color, we did not change it, but limited its use as a text color. Additionally, the styles.css source code provides defaults for different styles of text. This ensures that even if a style is not defined per element, there is a reference sheet so that contrast between components is good.

System Requirements Document (SRD)

W.E.S.S Network System Requirements Document:

Requirement Number	Child Nodes	Min	Max
1	Effective Communication Range	0Km	15Km
2	Power Requirements	0 - 0.3W @ 3V	0 - 0.3W @ 5V
3	Data Stream Period	0Min	Any (ideal 60min)
4	Effective Time Series Length	-	-
5	Required Regulatory	LoRaWAN	

	Compliances	FCC RF FCC Part 15.205 FCC Part 15.247 FCC Part 15.209 IEC 61140 IEC 60529 IEC 60664 IEC 60721 IEC 61010 RoHS - 173-337 WAC IEC 61000
6	Test Conditions	Temp: 25 °C ± 5 °C Humidity: 40% ± 10% Ambient EM interference: 0.19-0.43 V/m
7	Required Peripherals	REYAX RYLR998 Lora Communications module Gravity Analog TDS Sensor Gravity Analog Turbidity Sensor Jessinie MiCS-6814 gas sensor (NO2, CO, NH3)
8	Programming Language	C/C++

Table 1: Child Node System Requirements

Require ment Number	Parent Nodes	Min	Max
1	Effective Communication Range	0Km	15Km
2	Power Requirements	3.8W @ 6V	9.8W @ 6V
3	Data Stream Period	-	-

4	Effective Time Series Length	1-2 Weeks	3 Months
5	Required Regulatory Compliances	FCC Part 15.205 FCC Part 15.247 FCC Part 15.209 RFC 4180 ISO/IEC DIS 40500	
6	Test Conditions	Temp: 25 °C ± 5 °C Humidity: 40% ± 10% Ambient EM interference: 0.19-0.43 V/m	
7	Required Peripherals	REYAX RYLR998 Lora Communications module LCD touchscreen	
8	Programming Language	Python/CSS/Bash	

Table 2: Parent Node System Requirements

The system requirements for the W.E.S.S network:

[1]

Each W.E.S.S child node should be able to independently monitor environmental data (CO, NH₃, NO₂, TDS, TURB) and store an adjustable number of data samples to meet the transmission interval desired by the user. All data points collected within the specified time interval should be averaged according to their measurement type and sent to a parent node via LoRa communication modules.

[2]

Each W.E.S.S child node should be able to simultaneously communicate with the nearest parent node without interruption or loss of data.

[3]

The main W.E.S.S. hub is powered by a Raspberry Pi which can be powered via USB-C or battery power. Each child node can be implemented with a board of the user's choice, but one power requirement is that the RYLR998 LoRa modules used in our setup must

be powered by 3.3V VDD and logic. 5V will damage the LoRa modules and hence may require a level shifter if a board with 5V logic level signaling is used.

[4]

For optimal operation, the Pi and child modules should be kept at an ambient temperature above -40 degrees celsius and below +40 degrees celsius.

[5]

Each W.E.S.S. child node should communicate with the parent node via LoRa modules configured to the same frequency band (915 MHz for North America) and with the same network ID. The LoRa modules used are Reyax RYLR998 units which have an MCU capable of interfacing with the Raspberry Pi and other microcontrollers via UART. Usage of LoRa modules without UART interfaces is currently not supported.

[6]

The parent node requires Python 3.10 to be installed in order to run the webpage and LoRa scripts.

[7]

Code written for the child nodes should be able to fit within the on-chip flash and SRAM. Usage of external storage modules like SPI flash, eMMC, and microSD cards on child nodes is currently not supported. The Raspberry Pi should have at least an 8GB microSD card to fit Raspbian as well as the web interface program and any data received from child nodes. More nodes will require more storage.

Project Schedule

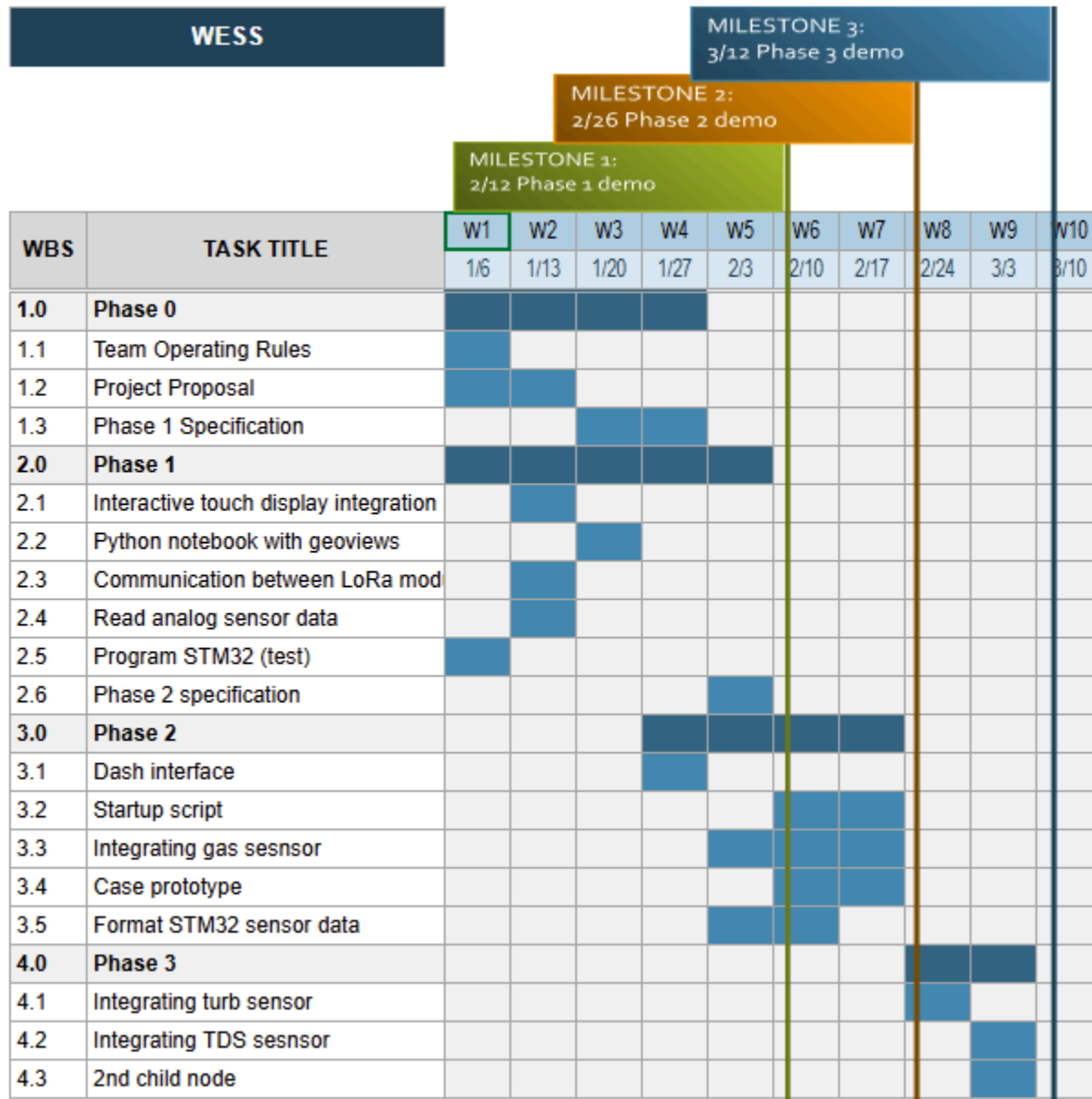


Fig. 1. Gantt Chart

Project Resources

The following resources were essential to the success of the W.E.S.S project:

- **Human Resources:**
 - A multidisciplinary team of six engineering students, each responsible for key areas (hardware, firmware, software, UI design, and testing).
- **Financial Resources:**
 - Total project funding capped at \$300, with strict budget management.
- **Equipment and Materials:**
 - STM32F4 Discovery board, Raspberry Pi 4, 3 x RYLR998 LoRa modules, various environmental sensors (Jissinie MiCS-6814 gas sensor, Gravity Total Dissolved Solids sensor, Gravity turbidity sensor), and 3D-printed enclosures.
- **Software and Tools:**
 - IAR Workbench, STM32Cube IDE, Python, and open-source web development libraries.
 - GitHub was utilized to save source code on a cloud server
 - An IDE with git integration is recommended
- **Facilities:**
 - The University of Washington embedded systems capstone laboratory, which provided access to prototyping tools (soldering stations, PCB assembly tools), and computer workstations for software development and testing.

Outline of Experiments

Since we developed our project in parallel teams, each group had its own experiments. Once we reached the end of the quarter we had cross-team experiments to ensure the model was fully functional.

Individual team experiments consisted of the following:

Team Experiments		
Data Analysis	Wireless	Data Acquisition
Test web page runtime for different packages and styles	STM32 USART functionality tested for communication with LoRa module; found that USART0 was infeasible for transmission; changed to USART3 which was able to transmit to LoRa with no issue	Functionality of RYLR998 LoRa module tested with simple string transmissions between child node and parent node. Again tested with longer more complex strings as sensor data became available.
Test time difference between child module transmission, parent module receiver, and web page update	Transmission of sensor readings from STM32 to LoRa module over USART3 was tested; success, confirmed by reception of data via other LoRa module connected to Arduino Nano ESP32 and then Raspberry Pi	Functionality and measurement sensitivity of Jissinie MiCS-6814 gas sensor tested first in clear, ambient environments with little pollutant concentrations, then tested again near an idling vehicle to measure the increase of pollutants of interest (CO, NH3, NO2)
Test limits of touchscreen navigation – <i>i.e.</i> what works and what doesn't when using touch as opposed to a mouse and keyboard	Formatting of readings transmitted over LoRa into parseable CSV format for reading and storage by Raspberry Pi tested; success.	Functionality of TDS and checked against purified water and old tap water (TDS sensor was able to reliably detect the difference between the two over several days proving reliable and consistent measurements)
Test user experience with page design, wording, and layout choices	Third node consisting of an Arduino Meg with an attached LoRa module and potentiometers to simulate "readings" tested and connected to the Raspberry Pi; success.	Functionality of turbidity sensor checked against clear water and murky water (turbidity sensor was able to reliably detect the difference between the two waters over several days proving reliable and consistent measurements)

Table 3: Team Experiment Overview

Once we began integration between the three team's components, experiments mostly consisted of ensuring that communications between devices were consistent. To test this, we

would allow the system to run for a few days. This allowed the child module to test the consistency of its sensors, the wireless LoRa modules to check if there was consistent data transfer, and the parent module to populate a CSV and gather real sample data to display on the webpage. This specific experiment was to be repeated several times to recheck all systems after major updates were applied to any of the systems.

Trial Designs

Bill Of Materials (BOM)

REYAX RYLR998 LoRa communications module ———> 3x (\$12.60 + tax)

Gravity Total Dissolved Solids sensor —————> 1x (\$17.00 + tax)

Gravity Turbidity sensor —————> 1x (\$14.90 + tax)

Jissinie MiCS-6814 multi gas sensor —————> 1x (\$26.95 + tax)

7" LCD capacitive touch screen —————> 1x (\$52.99 + tax)

Various PCB prototyping materials —————> 1x (\$59.38 + tax)

Total budget \$300

V.S

Total expenditures \$224.49

PHASE 1

The initial design consisted of interfacing the STM32F4Discovery board to one of the RYLR998 LoRa modules via UART, and the other LoRa module to be interfaced with the UART pins of an Arduino Nano ESP32 provided for testing purposes. The Nano was easier to set up via Arduino IDE to verify correct operation of the modules, the STM32 not so much as IAR Workbench can be a hassle to work with. Nevertheless, after consulting the documentation for both the LoRa modules as well as the STM32, we were able to transmit basic messages from the STM32's LoRa module to the one on the Nano, confirmed by the transmitted messages appearing in the latter's serial terminal.

The user interface in this phase shown in Figure [2] remained simple to validate the ability to read from a CSV file, as well as plot that information onto an interactive map. This design has the webpage title, map, and a text line that populates with the most recent node data when one is clicked on.

WESS Data



Fig. 2. Phase 1 User Interface

PHASE 2

The next design phase had us wiring the sensors to the STM32 and programming it to format and transmit the values over the LoRa interface. The gas sensor has a setup routine wherein it must be calibrated and stabilised before readings are taken and transmitted, and the TDS sensor also has its own calculation subroutine so as to get accurate readings of dissolved solids via measuring the water's electrical conductivity. The turbidity sensor outputs a voltage proportional to how clear the water is; a voltage at or near 5V indicates clear water whilst values closer to 0 indicate murkiness. The receiving module was switched from the Nano to the Pi, and a Python script to parse the received values into a .csv format was created.

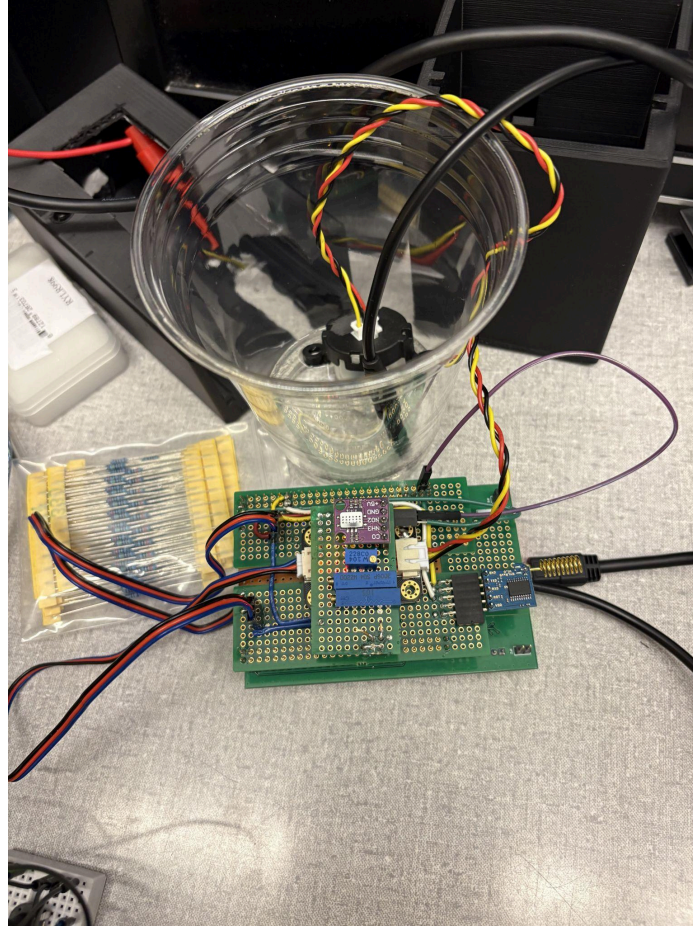


Fig. 3. Parent Node With Turbidity Sensor

In this phase the additional features required us to add tabs at the top of the screen to switch between different pages of the interface. The first tab retains the map interface with updates that show information when a node is hovered over with a mouse or a touchscreen hold. The nodes shown on the map are color coded according to the healthy range for each pollutant, selected using another row of tabs below the first. The Data Graphs page allows the user to select a node with a dropdown menu, a pollutant using the same row of tabs, and a time-series graph of that pollutant at that node.

WESS

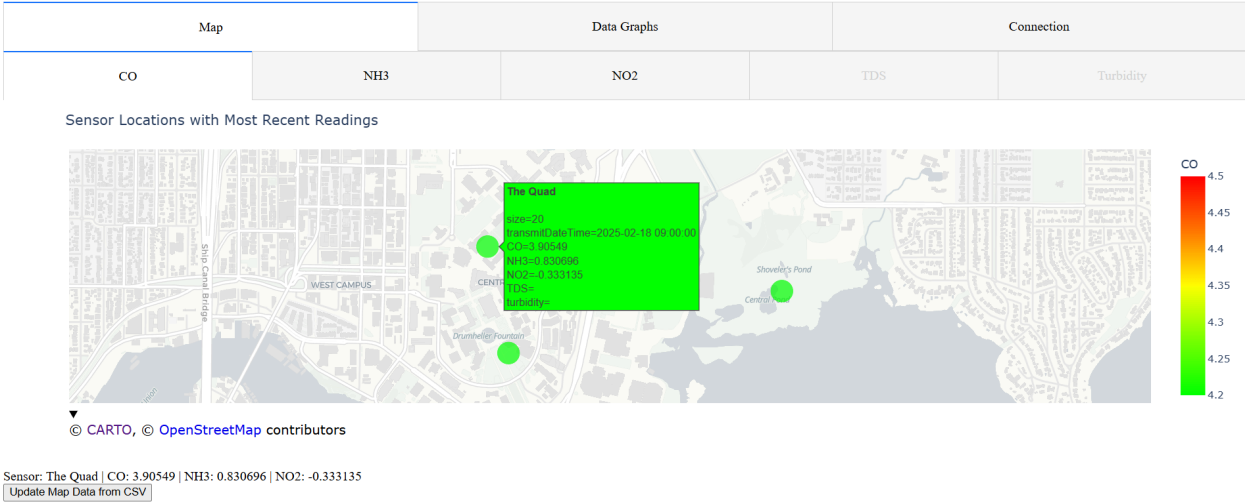


Fig. 4. Phase 2 Map Interface

WESS

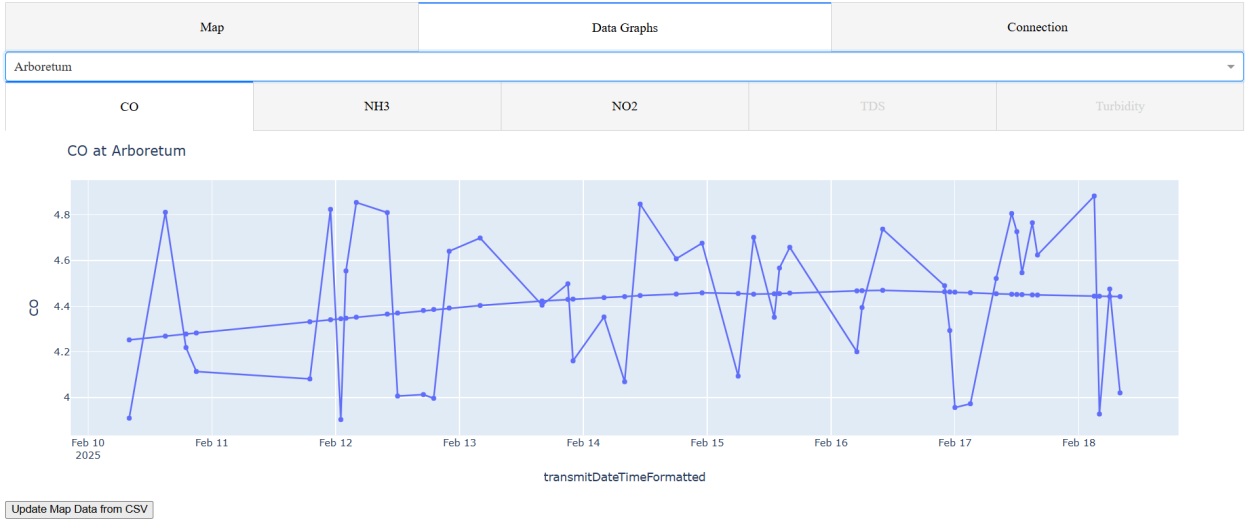


Fig. 5. Phase 2 Graph Interface

WESS

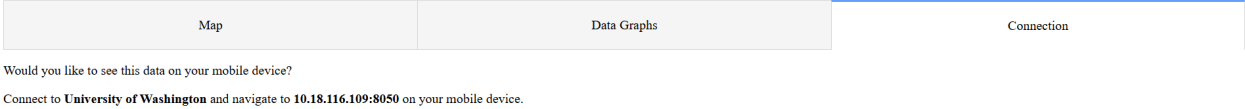


Fig. 6. Phase 2 Connection Interface

PHASE 3

Here, prototyping boards were used to neatly package and solder the wires and connectors required for the LoRa module and sensors, and the formatting of the CSV data from the STM32 was given timestamps for easier representation on the web-based visualisation tool. After this, a third “dummy” module based on an Arduino Mega was created with potentiometers to simulate the addition of a third node to the LoRa network, demonstrating our project’s scalability.

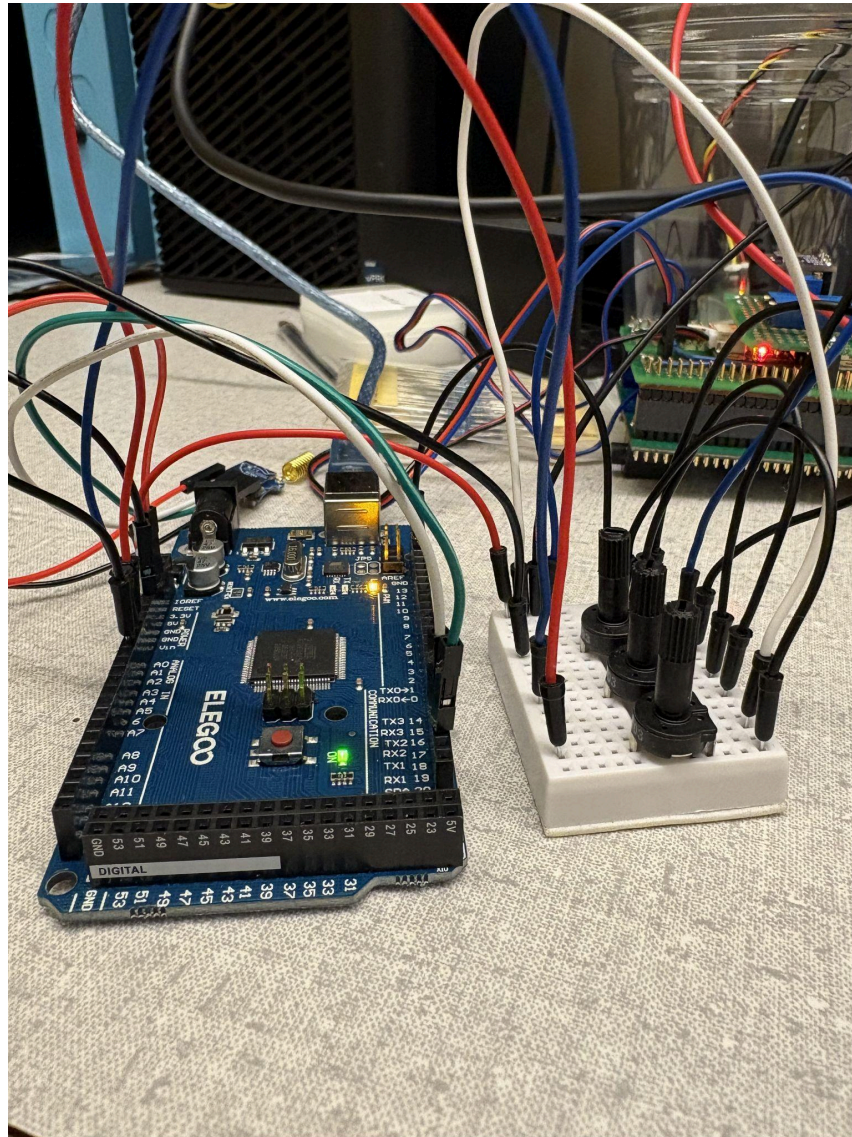


Fig. 7. Dummy Node

This phase added primarily visual refinement to the interface with CSS styling. Since the backbone of the webpage was complete and basic functionality was working, we had time and resources to make the page look appealing. One of the first changes from the Phase 2 design was adding a CSS stylesheet to format texts and backgrounds. This gave our web page a unique look to stand out against a minimal html page. Instead of a white background with black text we aimed for a ‘dark mode’ look with a dark background and light text. Additionally, the

sub-tabs for the Map and Trendline tabs were replaced with other interactive elements. The map page was replaced with a dropdown menu, since only one pollutant colorbar could be displayed at a time, so only one pollutant from the dropdown should be selectable at a given time. For the trendline interface, the drop down was arbitrarily swapped with a radio element. Instead of having a trendline for each pollutant rendered on separate tabs, they could now be toggled directly on the graph using the plotly library to express legend toggle features.

Both the map and trendline rely on hover features to display more information about the selected data point. The map was updated to only display the relevant pollutant value. The trendline hover data remained similar to what it was before but with some fine-tuning to make it look more appealing. Similar small tweaks were applied to the html elements until we had our final Phase 3 design.

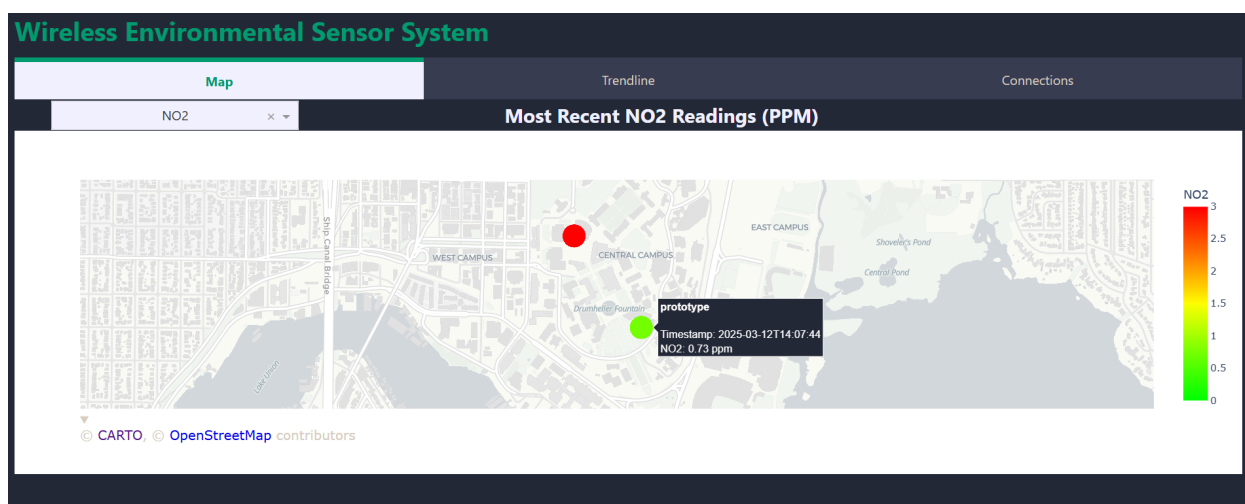


Fig. 8. Phase 3 Map Interface

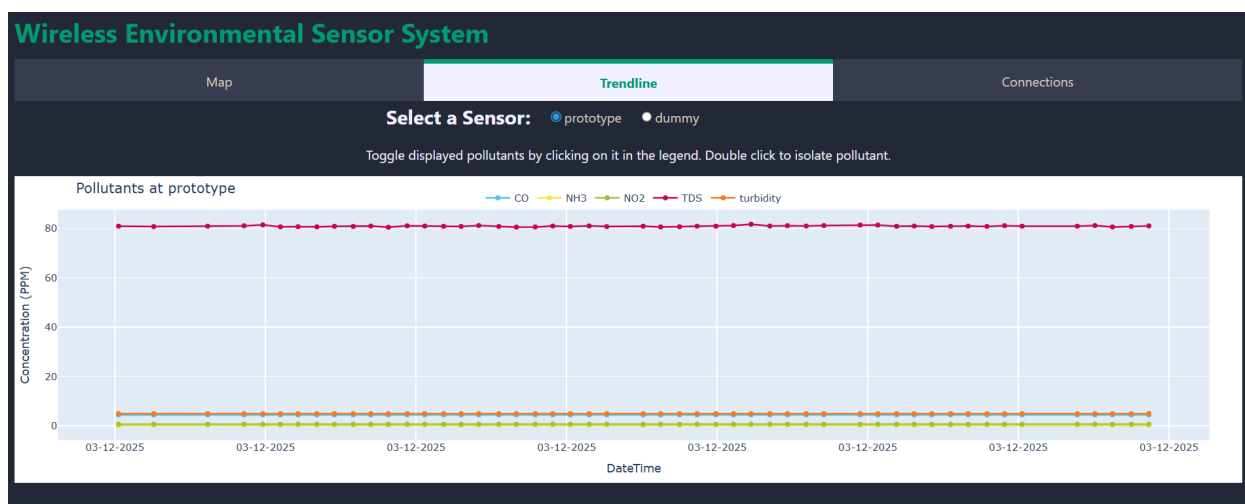


Fig. 9. Phase 3 Trendline Interface

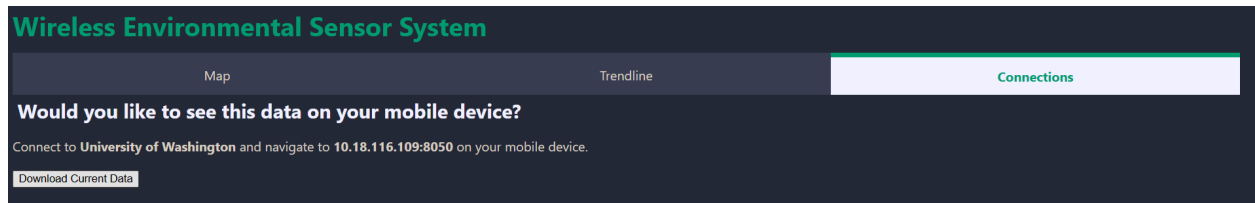


Fig. 10. Phase 3 Connection Interface

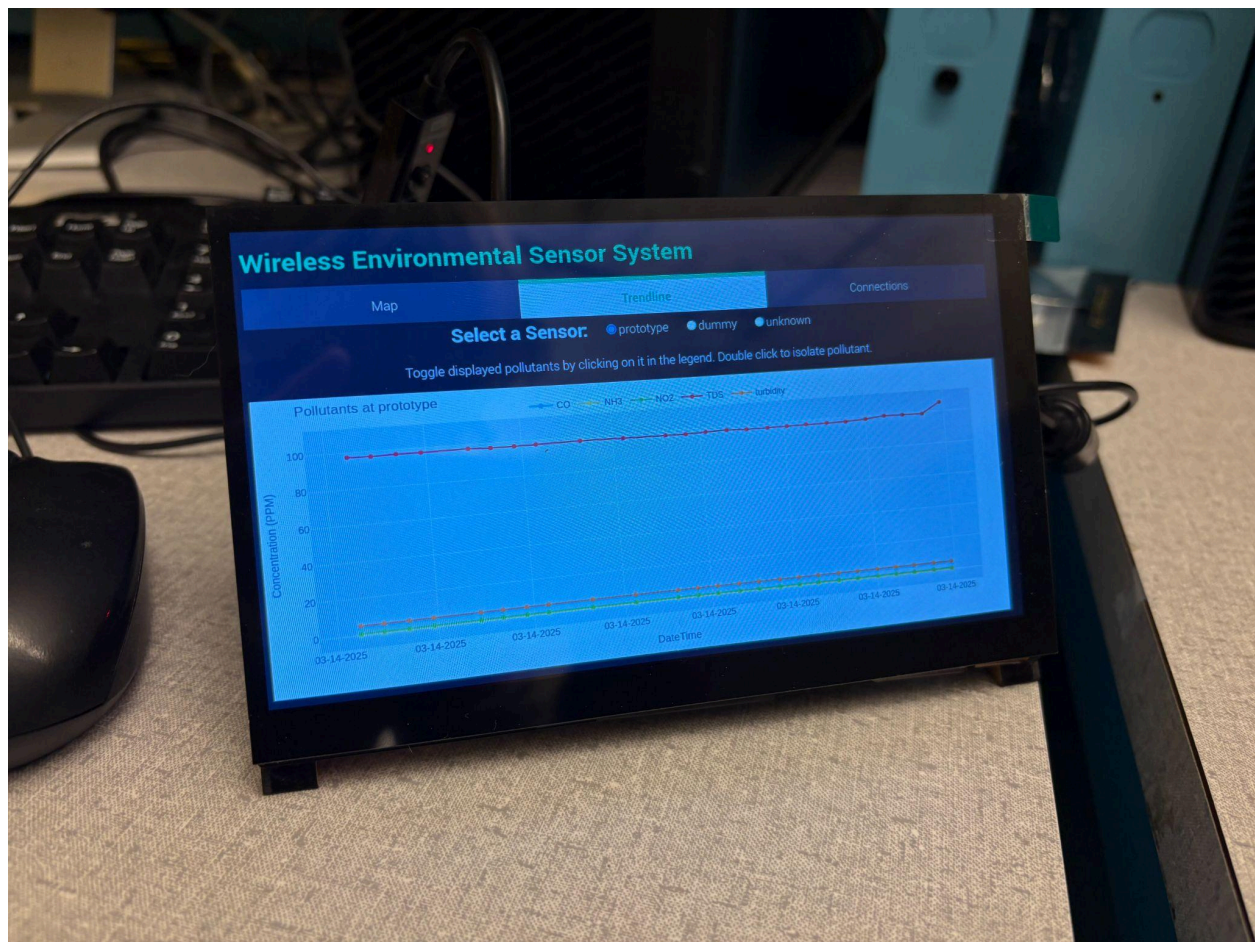


Fig. 11. WESS Interface Running on Screen

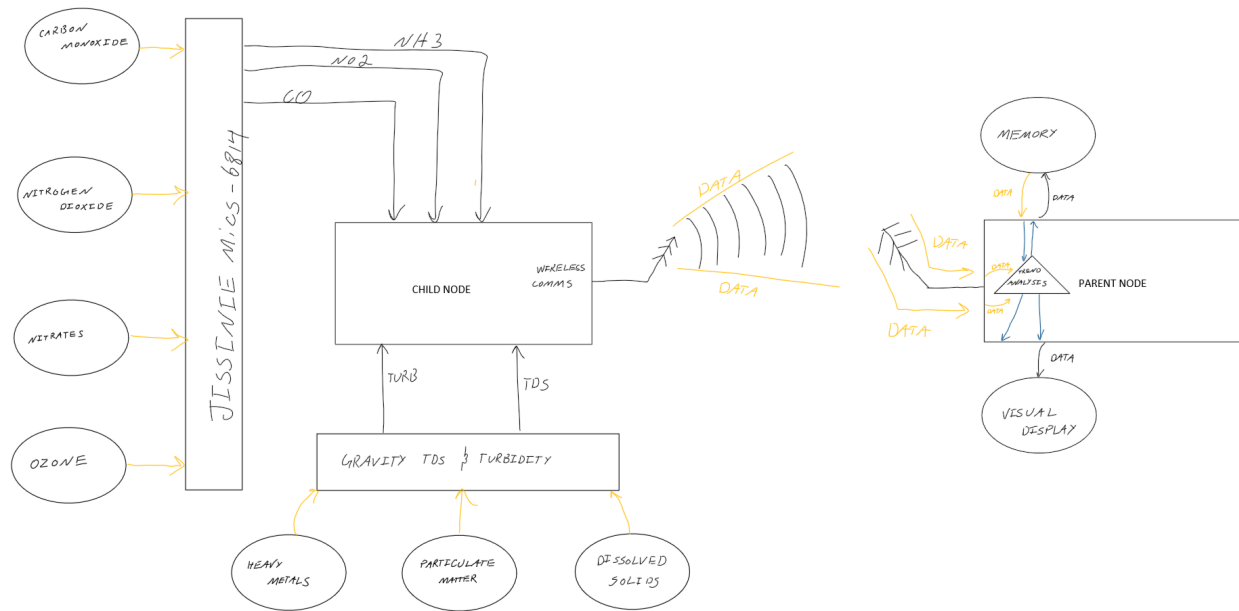


Fig. 12. Top level functionality block diagram

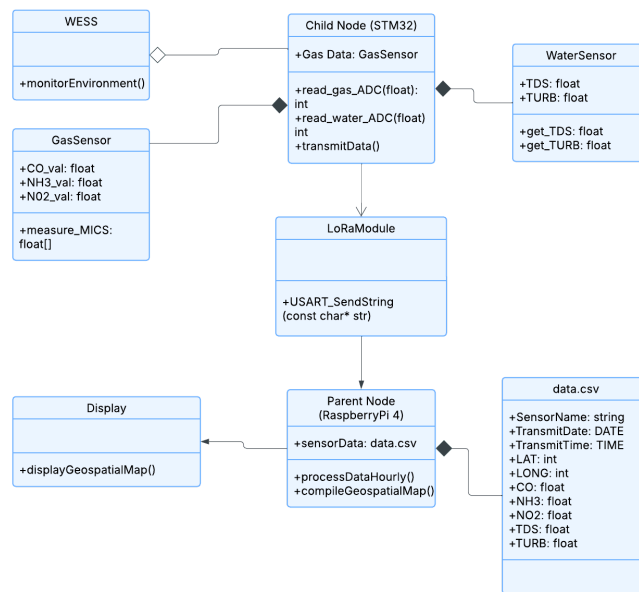


Fig. 13. W.E.S.S UML chart

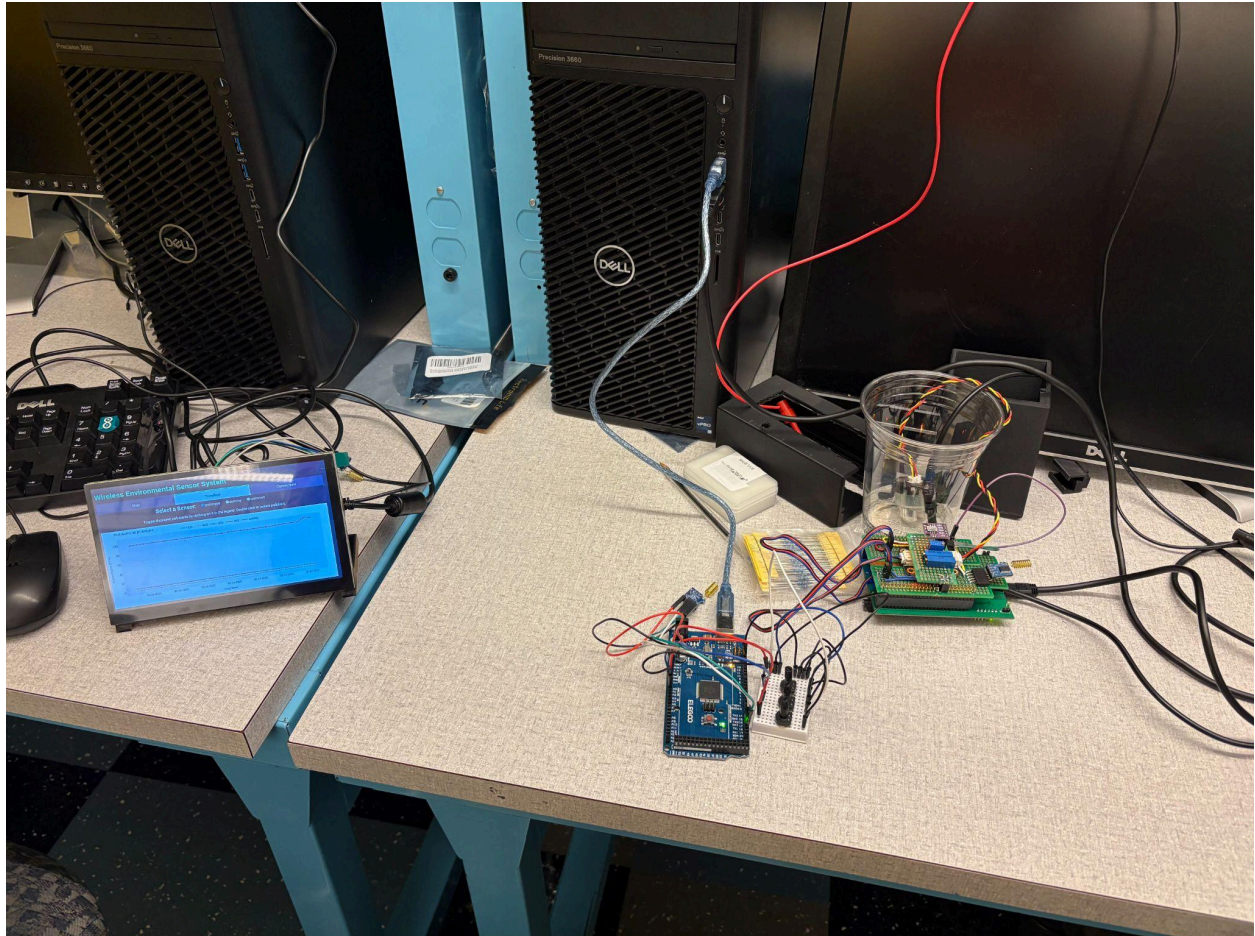


Fig. 14. Completed WESS Setup

Experimental Outcomes

In Phase 1, we initially tried to use USART0 for communicating with the LoRa module, but transmissions were not coming through despite explicit formatting and consulting the documentation of the module. Using a USB-TTL debugger, we discovered that USART0 has a capacitor on one of its data lines that tends to garble whatever data is sent to it. After switching our interface to USART3, interfacing with the LoRa module worked without a hitch.

Additionally, another issue we faced while developing the prototype for the child node came from the general lack of documentation and interfacing information on the datasheet for the Jissinie MiCS-6814 gas sensor. This made it difficult to get proper gas measurements from the MiCS sensor since there was no guide on how to convert the analog reading from the STM32's ADC pins to gas concentrations. To remedy this, we implemented a third-party hardware configuration with pull-up resistors on the reading channels and with conversion equations from a functional Arduino library for the MiCS sensor created by Lukija ([MICS6814-lukija.ino](https://github.com/lukija/MICS6814-lukija)) [13].

We needed to adapt the Arduino code since it was not directly compatible with the STM32 but after some work and testing, we were able to get readings from the MiCS gas sensor.

Once we were able to get the gas sensor integrated with the STM32 we added the TDS sensor and configured the STM32's ADC channels to regularly read the data lines connected to these sensors. To test the functionality of the TDS sensor we submerged it in a small cup of purified water to get a baseline measurement and was then compared to a solution of old tap water. As expected, the solution of tap water gave a much higher TDS reading than the purified water, this test was done again over several days to verify readings and stability of the sensor output and every time the readings fell within similar ranges of one another.

To check the functionality of the gas sensors we allowed them to undergo a digital calibration stage to gather ambient gas levels and provide a reference point we could compare to later. After calibration, we made use of one of our vehicles and allowed the engine to idle and produce exhaust fumes that were known to contain CO, NH₃, and NO₂ or some trace amount. We then brought the sensor assembly close to the exhaust pipe to see if the sensor outputs would change for CO, NO₂, or NH₃ and what we discovered was that the sensor was able to pick up on the change in concentration of each of the gasses although CO was the most prominent change as we anticipated. The turbidity sensor was not integrated into the design until the end of phase 2 but its testing was identical to that of the TDS sensor except with clear water and a murkier water and its readings came back as expected as well.

A similar phenomenon happened in Phase 2 when we switched the receiver from the Arduino Nano to the Raspberry Pi. We had connected it to the LoRa module via UART0 and were able to receive data, however, it was also accompanied by error messages from the LoRa module and would stop working after 5 seconds. UART0 is where the Raspberry Pi puts its terminal emulator output by default, which sends conflicting signals to the LoRa module. This had to be disabled, and once done, LoRa transmission between the Pi and STM32 worked seamlessly.

Experimental outcomes of the front-end development were very qualitative and sometimes subjective. Initial web page development began with looking at the Python package Geoviews since there was previous familiarity with it. However, after working with the package to get a basic webpage for our Phase 1 demonstration, we realized that the script ran extremely slow on the Raspberry Pi due to the tile rendering running after tiles were downloaded from the internet. We pivoted on this package to another Python package, Dash. Dash is a package that allows HTML and CSS code to be written in a Python script. Switching to Dash over geoviews reduced our load times and as an unexpected bonus gave us increased versatility in design tools.

After the switch to building with Dash, most of the web page 'experimental' outcomes came from layout and element changes. This involved 'playing around' with the different available HTML elements and CSS styles. Another issue we ran into in phase 1 was the lack of any way to update the map data after receiving changes in the CSV. The only way around this at this point was to restart the app entirely, which we did not want to be necessary for the user to do.

In Phase 2 we relied heavily on the tab feature to have different states for the figures. This was functional but used a lot of our valuable vertical page real estate on the relatively small screen the Pi was attached to. Additionally we ran into a design choice issue with the “Update map data from CSV” button. It was confusing from a wording standpoint (some users may not be familiar with the fact we were storing the data in a CSV file) as well as a design standpoint (users assumed the map would automatically update in the first place).

When we began work on our Phase 3 design, we replaced some of the tabs with drop-down menus and radio items. This reduced the amount of vertical space required while still giving the user control over what data is displayed. Additionally we made the map and graph tabs automatically reload data from the CSV every few seconds, so the need for manually refreshing those pages was unnecessary.

Impact and Consequences

The impact and consequences of the W.E.S.S project extend both locally and globally. Locally, the system provides communities, public health agencies, and environmental conservation groups with actionable real-time data that can lead to better-informed decisions and timely responses to pollution events. By offering precise measurements of air and water quality, W.E.S.S empowers municipal planners and researchers to target interventions, optimize urban planning, and enhance environmental safety. On a broader scale, the network serves as a model for sustainable monitoring solutions, demonstrating how scalable and cost-effective technologies can support global initiatives aimed at reducing pollution and mitigating environmental risks. This project not only enhances the technical capabilities of environmental data collection but also fosters a collaborative approach to addressing the complex challenges of public health and environmental conservation.

Conclusions and Recommendations

In conclusion, the W.E.S.S network has proven its technical viability and operational robustness through iterative testing and real-world deployments. The project successfully validated a comprehensive solution for continuous environmental monitoring, from sensor integration and wireless communication to real-time data visualization and analysis. Key lessons learned include the importance of adaptive communication protocols and rigorous calibration processes, which have informed both the current design and future iterations of the system. Moving forward, recommendations include expanding the range of monitored parameters, integrating advanced data analytics for predictive insights, and exploring alternative power solutions to enhance sustainability and portability. Overall, the W.E.S.S project lays a solid foundation for next-generation environmental monitoring systems that can scale from individual homes to entire cities, thereby contributing significant value to both technological innovation and environmental stewardship.

References, Acknowledgements, and Intellectual Property

References:

- [1] *RADIO FREQUENCY DEVICES*, 47 CFR 15, March 2025. Available: <https://www.ecfr.gov/current/title-47/chapter-I/subchapter-A/part-15>
- [2] PROTECTION AGAINST ELECTRIC SHOCK, IEC 61140, January 2016. Available: <https://cdn.standards.iteh.ai/samples/20618/af14db91191b436fa5a33bd5ef4c918e/IEC-61140-2016.pdf>
- [3] DEGREES OF PROTECTION PROVIDED BY ENCLOSURES (IP Code), IEC 60529, February 2001. Available: <https://www.wewontech.com/IEC60529-%20IP-Standard.pdf>
- [4] INSULATION COORDINATION FOR EQUIPMENT WITHIN LOW-VOLTAGE SUPPLY SYSTEMS, IEC 60664, May 2020. Available: <https://cdn.standards.iteh.ai/samples/100156/f010cc07edff461d9caa559635817555/IEC-60664-1-2020.pdf>
- [5] CLASSIFICATION OF ENVIRONMENTAL CONDITIONS, IEC 60721, May 2019. Available: <https://cdn.standards.iteh.ai/samples/101155/a68fe74bf9d240e0976efac88ee4382c/IEC-60721-3-3-2019.pdf>
- [6] Safety requirements for electrical equipment for measurement, control, and laboratory use, IEC 61010, December 2016. Available: <https://cdn.standards.iteh.ai/samples/22220/a6c5cee6104548a68e3719a79066c2c4/IEC-61010-1-2010-AMD1-2016.pdf>
- [7] RP002-1.0.3 LoRaWAN® Regional Parameters, May 2021. <https://lora-alliance.org/wp-content/uploads/2021/05/RP002-1.0.3-FINAL-1.pdf>
- [8] Reyax, RYLR998 UART Interface 868/915 MHz Lora Antenna Transceiver Module Datasheet, July 2022. <https://fcc.report/FCC-ID/QLYRYLR998/6024791.pdf>
- [9] STM32F4Discovery Header Files, Jan 2025. Files provided via Canvas by course staff.
- [10] Safer Products Restrictions and Reporting, 173-337 WAC, May 2024. <https://ecology.wa.gov/regulations-permits/laws-rules-rulemaking/closed-rulemaking/wac-173-337-may2023#:~:text=On%20May%2031%2C%202023%2C%20Ecology,as%20directed%20by%20Chapter%2070A.>
- [11] Common Format and MIME Type for Comma-Separated Values (CSV) Files, RFC 4180, October 2005. <https://www.ietf.org/rfc/rfc4180.txt>
- [12] Web Content Accessibility Guidelines (WCAG) 2.1, December 2024. <https://www.w3.org/TR/2024/REC-WCAG21-20241212/#abstract>
- [13] Lukija, "CJMCU-6814 adapter board with MICS-6814 CO/NH3/NO2-sensor" forum.arduino.cc. Available: <https://forum.arduino.cc/t/cjmcu-6814-adapter-board-with-mics-6814-co-nh3-no2-sensor/595612>
- [14] yuyouliang, "GravityTDS" github.com, Jan 2018. Available: <https://github.com/DFRobot/GravityTDS/tree/master>

The following Python packages were procured from PyPi via Pip and used at varying points throughout development:

anyio==4.8.0
argon2-cffi==23.1.0
argon2-cffi-bindings==21.2.0
arrow==1.3.0
asttokens==3.0.0
async-lru==2.0.4
attrs==25.1.0
babel==2.16.0
beautifulsoup4==4.12.3
bleach==6.2.0
blinker==1.9.0
bokeh==3.6.2
Cartopy==0.24.1
certifi==2025.1.31
cffi==1.17.1
charset-normalizer==3.4.1
click==8.1.8
colorcet==3.1.0
comm==0.2.2
composites==0.7.2
contourpy==1.3.1
coverage==7.6.10
coveralls==4.0.1
cyclr==0.12.1
Cython==3.0.11
dash==2.18.2
dash-core-components==2.0.0
dash-html-components==2.0.0
dash-table==5.0.0
datashader==0.17.0
debugpy==1.8.12
decorator==5.1.1
defusedxml==0.7.1
docopt==0.6.2
executing==2.2.0
fastjsonschema==2.21.1
Flask==3.0.3
fonttools==4.55.8
fqdn==1.5.1

future==1.0.0
geoviews==1.14.0
h11==0.14.0
holoviews==1.20.0
httpcore==1.0.7
httpx==0.28.1
hvplot==0.11.2
idna==3.10
importlib_metadata==8.6.1
ipykernel==6.29.5
ipython==8.32.0
ipywidgets==8.1.5
iso8601==2.1.0
isoduration==20.11.0
itsdangerous==2.2.0
jedi==0.19.2
Jinja2==3.1.5
json5==0.10.0
jsonpointer==3.0.0
jsonschema==4.23.0
jsonschema-specifications==2024.10.1
jupyter==1.1.1
jupyter-console==6.6.3
jupyter-events==0.11.0
jupyter-lsp==2.2.5
jupyter_client==8.6.3
jupyter_core==5.7.2
jupyter_server==2.15.0
jupyter_server_terminals==0.5.3
jupyterlab==4.3.5
jupyterlab_pygments==0.3.0
jupyterlab_server==2.27.3
jupyterlab_widgets==3.0.13
kiwisolver==1.4.8
linkify-it-py==2.0.3
llvmlite==0.44.0
Markdown==3.7
markdown-it-py==3.0.0
MarkupSafe==3.0.2
matplotlib==3.10.0
matplotlib-inline==0.1.7
mdit-py-plugins==0.4.2

mdurl==0.1.2
mistune==3.1.1
multipledispatch==1.0.0
narwhals==1.26.0
nbclient==0.10.2
nbconvert==7.16.6
nbformat==5.10.4
nest_asyncio==1.6.0
notebook==7.3.2
notebook_shim==0.2.4
numba==0.61.0
numpy==2.1.3
overrides==7.7.0
packaging==24.2
pandas==2.2.3
pandocfilters==1.5.1
panel==1.6.0
panels==0.3.1
param==2.2.0
parso==0.8.4
patsy==1.0.1
pexpect==4.9.0
pillow==11.1.0
platformdirs==4.3.6
plotly==6.0.0
prometheus_client==0.21.1
prompt_toolkit==3.0.50
psutil==6.1.1
ptyprocess==0.7.0
pure_eval==0.2.3
pycparser==2.22
pyct==0.5.0
Pygments==2.19.1
pyparsing==3.2.1
pyproj==3.7.0
pyserial==3.5
pyshp==2.3.1
python-dateutil==2.9.0.post0
python-json-logger==3.2.1
pytz==2025.1
pyviz_comms==3.0.4
PyYAML==6.0.2

pyzmq==26.2.1
referencing==0.36.2
requests==2.32.3
retrying==1.3.4
rfc3339-validator==0.1.4
rfc3986-validator==0.1.1
rpds-py==0.22.3
scipy==1.15.1
Send2Trash==1.8.3
serial==0.0.97
setuptools-git-version==1.0.3
shapely==2.0.7
six==1.17.0
sniffio==1.3.1
soupsieve==2.6
stack-data==0.6.3
statsmodels==0.14.4
structsolve==0.2.2
terminado==0.18.1
tinycss2==1.4.0
toolz==1.0.0
tornado==6.4.2
tqdm==4.67.1
traitlets==5.14.3
types-python-dateutil==2.9.0.20241206
typing_extensions==4.12.2
tzdata==2025.1
uc-micro-py==1.0.3
uri-template==1.3.0
urllib3==2.3.0
wcwidth==0.2.13
webcolors==24.11.1
webencodings==0.5.1
websocket-client==1.8.0
Werkzeug==3.0.6
widgetsnbextension==4.0.13
xarray==2025.1.2
xyzservices==2025.1.0
zipp==3.21.0